

Semesterprojekt WS24

Entwicklung einer Schichtplanungs-Webapplikation (V1.0)

Entwickeln Sie eine Webapplikation zur Verwaltung von Schichtplänen in verschiedenen Dienststellen (z. B. Krankenhäuser, Restaurants). Die Applikation soll folgende Funktionalitäten bieten, die sich in drei Benutzerrollen unterteilen: Administrator:innen, Manager:innen und Mitarbeiter:innen.

Allgemeine Funktionalitäten

Eine **Dienststelle** beschäftigt Mitarbeiter:innen und hat vorgegebene Dienstzeiten. Eine Dienststelle kann beliebig granular sein. Größere Organisationen können zum Beispiel aus mehreren Dienststellen bestehen, um die Übersichtlichkeit zu gewährleisten.

Beispiel: Ein kleines Bistro beschäftigt 3 Mitarbeiter:innen und hat an Wochentagen von 9 bis 16 Uhr geöffnet. Das Blutlabor der örtlichen Klinik beschäftigt 10 Mitarbeiter:innen und ist rund um die Uhr zu besetzen.

Hinweis: Organisationen (übergeordnet zu Dienststellen) werden in unserer Anwendung nicht berücksichtigt und müssen nicht implementiert werden.

Ein **Schichtplan** ist einer Dienststelle zugeordnet und für jede Woche wird ein eigener Schichtplan erstellt. Ein Schichtplan kann beliebig viele Schichten beinhalten.

Beispiel: Der Schichtplan des Bistros hat 5 Schichten, je eine pro Wochentag, von 8:30 bis 16:30 Uhr. Das Blutlabor hat 3×7 Schichten, d. h. pro Tag 3 Schichten.

Ein **Schichtplan** kann im Status „in Entwurf“, „veröffentlicht“ oder „deaktiviert“ sein und pro Woche kann es maximal einen veröffentlichten Schichtplan geben.

Hinweis: Es bleibt Ihnen überlassen, ob Sie pro Woche ebenfalls nur maximal einen „Entwurfs“-Schichtplan erlauben.

Eine **Schicht** zeigt an, wann Mitarbeiter:innen arbeiten. Jede Schicht hat einen Start- und Endzeitpunkt und kann beliebig viele Mitarbeiter umfassen.

Beispiel: Im Bistro ist jede Schicht normalerweise mit 2 Mitarbeiter:innen belegt. Im Blutlabor sind Tagesschichten üblicherweise mit 3 Mitarbeiter:innen belegt, Nacht- und Wochenendschichten mit 2 Mitarbeiter:innen.

Hinweise:

- Das Teilen von Schichten muss nicht berücksichtigt werden. (Bsp.: Die erste Schichthälfte wird von Mitarbeiter 1, die zweite von Mitarbeiter 2 belegt)
- Mindestanforderungen an Schichten müssen nicht berücksichtigt werden. (Bsp.: im Bistro mindestens 2 Personen pro Schicht arbeiten)

Entstehung eines Schichtplans

Der Workflow zur Erstellung und Veröffentlichung eines Schichtplans umfasst folgende Schritte:

1. **Erstellung:** Manager:innen erstellen oder bearbeiten einen Schichtplan für ihre Organisationen. Der Schichtplan wird zunächst als Entwurf gespeichert, auf den Mitarbeiter:innen keine Einsicht haben. Manager:innen können den Entwurf anpassen, Schichten hinzufügen oder entfernen sowie Mitarbeiter:innen zuweisen oder entfernen.
2. **Abwesenheiten:** Mitarbeiter:innen können Abwesenheiten eintragen, d. h. Zeiten, in denen sie nicht zur Verfügung stehen. Diese Abwesenheiten können einmalig oder regelmäßig wiederkehrend sein.

3. **Validierung:** Vor der Veröffentlichung wird der Entwurf auf Konsistenz geprüft, insbesondere hinsichtlich der Verfügbarkeiten der Mitarbeiter:innen und möglicher Konflikte (z. B. eine Mitarbeiterin wurde in zwei Schichten zur gleichen Zeit in verschiedenen Dienststellen eingetragen, oder hat eine geplante Abwesenheit). Alle Konflikte müssen behoben werden, bevor der Schichtplan veröffentlicht werden kann.
4. **Veröffentlichung:** Nach erfolgreicher Prüfung wird der Schichtplan veröffentlicht und ist für die Mitarbeiter:innen sichtbar. Ab diesem Zeitpunkt ist der Schichtplan final und kann nicht mehr geändert werden. Abwesenheiten, die nach der Veröffentlichung eingetragen werden, haben keinen Einfluss auf den bestehenden Schichtplan.
5. **Wiederverwenden von Plänen:** Manager:innen können Schichtpläne aus Vorwochen kopieren, um diese für eine zukünftige Woche zu verwenden. Dabei werden alle Schichten mit den zugewiesenen Mitarbeiter:innen übernommen und in die zukünftige Woche verschoben. Der kopierte Dienstplan wird als Entwurf gespeichert.

Kurzfristige Anpassungen

Bei unvorhersehbaren Ereignissen (z. B. Krankheit oder Sonderurlaub eines Mitarbeiters) können Manager:innen wie folgt vorgehen:

1. **Neuer Entwurf:** Die Manager:in erstellt eine neue Version des bestehenden Schichtplans als Entwurf und nimmt die erforderlichen Anpassungen vor.
2. **Deaktivieren des alten Schichtplans:** Der veröffentlichte, aber ungültige Schichtplan wird manuell deaktiviert.
3. **Veröffentlichung des angepassten Schichtplans:** Der geänderte Schichtplan durchläuft erneut den Veröffentlichungsprozess inklusive Validierung (siehe oben). Sofern keine Konflikte auftreten, wird dieser veröffentlicht.

Hinweise:

- **Mitarbeiter:innen müssen unvorhersehbare Ereignisse persönlich an ihre Manager:in melden. Die Anwendung muss keine Krankmeldungen oder Urlaubsanträge unterstützen.**
- **Dieser Ablauf muss als UML-Sequenzdiagramm modelliert werden (siehe Termine und Fristen).**

Feiertage

Feiertage müssen nicht beachtet werden, weder in der Darstellung noch für Abwesenheiten oder Standardöffnungszeiten. Sie können davon ausgehen, dass Manager:innen ihre Mitarbeiter:innen nicht an Feiertagen einteilen.

Rollen und deren Funktionalitäten

Administrator

Administrator:innen übernehmen die Systemverwaltung. Sie haben folgende Aufgaben:

- Benutzer:innen erstellen, bearbeiten und löschen.
- Rollen (Administrator, Manager, Mitarbeiter) vergeben und entziehen.
- Dienststellen erstellen, bearbeiten und löschen.
- Manager:innen Dienststellen zuweisen, Zuweisungen ändern oder entziehen.

Manager

- Wöchentliche Schichtpläne für die zugewiesenen Dienststellen erstellen und bearbeiten.
- Mitarbeiter:innen unter Berücksichtigung ihrer Verfügbarkeit Schichten zuweisen. (Schichten werden erst im Veröffentlichungsworkflow geprüft).

- Wöchentliche Dienstpläne nach erfolgreicher Prüfung veröffentlichen.
- Für kurzfristige Änderungen, können Manager:innen weitere Dienstpläne „in Entwurf“ erstellen, ungültige Dienstpläne deaktivieren und den neuen Veröffentlichungsprozess erneut starten
- Manager können alle Mitarbeiter im System sehen, um eine dienststellenübergreifende Planung zu ermöglichen.

Hinweis: Sie können davon ausgehen, dass Manager:innen verantwortungsbewusst handeln und Mitarbeiter:innen nur dann Schichten zuweisen, wenn sie tatsächlich an der entsprechenden Dienststelle tätig sind.

Mitarbeiter

- Einsicht in den persönlichen Arbeitsplan, sobald dieser veröffentlicht wurde.
- Angabe der Verfügbarkeiten (Tage und Uhrzeiten), die bei der Schichtplanung berücksichtigt werden.

Schichtplan-Darstellung

Der Schichtplan wird in einer Kalenderansicht dargestellt, die die gesamte Woche übersichtlich zeigt. Die Ansicht enthält Schichten mit Details wie Start- und Endzeiten sowie den zugewiesenen Mitarbeitern. Manager können den gesamten Plan einsehen und die Ansicht auf einzelne Dienststellen filtern, während Mitarbeiter nur ihre eigenen Schichten sehen können. Farbmarkierungen helfen, Schichten und eventuelle Konflikte schnell zu identifizieren.

Allgemeine Anforderungen

Für die Abgabe des Projekts müssen alle beschriebenen Funktionalitäten implementiert werden und das System lauffähig sein. Zusätzlich werden an das System folgende allgemeine Anforderungen gestellt:

- Die Plattform soll mit den aktuellen Browsern kompatibel sein.
- Alle Listenansichten sollen filter- und sortierbar sein.
- Auf unerwartete Systemzustände (Exceptions) soll das System angemessen reagieren und der Anwender:in entsprechende Fehlermeldungen zurückliefern. Eine generische Fehlermeldung, wie z. B. „ein unerwarteter Fehler ist aufgetreten“, ist für erwartbare Ereignisse unzureichend.
- Das System soll benutzerfreundlich (Usability) gestaltet und intuitiv bedienbar sein. Ihr System soll so reagieren, wie Sie es von einem realen Dienst erwarten.
- Das System soll multiuserfähig sein: Jeder User des Systems soll mit Username, Passwort (encoded), Vorname, Nachname, Rolle und E-Mail-Adresse gespeichert werden. User müssen sich immer erfolgreich anmelden, um Zugriff auf das System zu erhalten.
- Das Löschen von Entitäten (Benutzer, Dienststellen, ...) soll unterstützt werden. Dabei ist zu beachten, dass das Löschen von bestimmten Entitäten weitere Auswirkungen auf das System haben kann (z. B. wenn eine Nutzer:in gelöscht wird, was passiert mit deren Schichten?). Hierfür sollen geeignete „Lösch-Policies“ definiert und umgesetzt werden.

Technische Anforderungen

Die Anwendung muss in einer 3-Schichten-Architektur entwickelt werden. Verwenden Sie das Spring Framework, insbesondere Spring Core, Spring Data und JPA.

Hinweis: In der Vorlesung wurde Ihnen ein Demo-Skeleton-Projekt vorgestellt, welches als UI React verwendet. Bevorzugt ist dieses Projekt als Basis zu verwenden, da es für Sie (1) bereits eine vollständige Grundstruktur bietet, und (2) wir Sie in dieser Konfiguration optimal bei der Entwicklung unterstützen können. Grundsätzlich steht es Ihnen aber frei, ein anderes UI-Framework zu wählen. Sie müssen allerdings in der Lage sein, dieses

selbstständig mit Spring zu konfigurieren. Bei etwaigen Fragen werden wir uns zwar bemühen Ihnen zu helfen, wir können aber keine Unterstützung garantieren.

Das umfangreiche und automatisierte Testen einer Applikation ist essenziell. Testen Sie Ihren Source-Code mit realitätsnahen Testdaten in ausreichender Qualität und Quantität durch **Unit- und Integration-Tests**. Verwenden Sie SonarQube, um die Qualität Ihres Quellcodes regelmäßig zu überprüfen. Achten Sie außerdem darauf, die gefundenen Issues (Bugs, Vulnerabilities, Code-Smells, ...) zeitnah zu beheben.

Hinweis: Automatisierte UI- und API-Tests sind nicht erforderlich. Das bedeutet, dass Sie keine Tests für Frontend-Code schreiben müssen. Ebenfalls müssen Sie die API nicht mit Postman o. Ä. testen. Es macht dennoch Sinn, das Frontend durch SonarQube untersuchen zu lassen, um über Code-Smells informiert zu bleiben.

Organisatorisches und Entwicklungsrichtlinien

Die Projektarbeit erfolgt im Team (4-5 Personen). Wir ersuchen Sie im Proseminar am 9.12.2024 Ihr Team und jeweils eine Ansprechperson Ihrer Lehrveranstaltungsleitung mitzuteilen.

Folgende weiteren Punkte sind verpflichtend umzusetzen:

- Die LV-Leitung wird Ihnen ein GIT-Repository zur Verfügung stellen. Legen Sie alle Ihre erstellten Artefakte (z. B. Quellcodedateien, Diagramme, ...) in diesem GIT Repository ab.
- Verwenden Sie das im Laufe des Semesters vorgestellte GIT Branching Modell mit main, dev und Feature-Branches.
- Halten Sie sich bei der Programmierung an die in der Vorlesung besprochenen Java Code Conventions.
- Benennen Sie Commit-Messages sinnvoll.
- Achten Sie auf eine sinnvolle und aussagekräftige Dokumentation ihres Sourcecodes (Eine automatische Generierung von JavaDoc Kommentaren ist nicht zielführend!). Grundsatz: Ein neuer Mitarbeiter sollte sich anhand der Dokumentation in das Projekt einarbeiten können.

Weitere wichtige Hinweise

- Sollten Sie Fragen zur Aufgabenstellung oder dem Technologieeinsatz haben, stellen Sie diese bitte im OLAT Forum.
- Im Normalfall werden Punkte einheitlich für das gesamte Team vergeben. Stellt sich jedoch heraus, dass sich einzelne Studierende besonders viel oder wenig im Team einbringen, kann auch eine vom restlichen Team unabhängige Punkteanzahl vergeben werden.
- Die Arbeitsleistung der einzelnen Team-Mitglieder wird AUSSCHLIESSLICH durch direkte Beiträge (Commits, Code Contributions) festgestellt. Pair-Programming bei dem nur eine Person programmiert wird ausschließlich für die Person gewertet die commitet. Beachten Sie, dass sich die Commit-Autor:in beim Überschreiben von Commits (durch rebase / squash, aber auch merge / squash) ändern kann. Auch in diesem Fall wird der Commit der Autor:in angerechnet die zum jeweiligen Abgabzeitpunkt Hauptautor:in ist.
- Die Teamarbeit steht bei diesem Projekt im Vordergrund, die Lehrveranstaltungsleiter achten besonders auf die gleichmäßige Verteilung von Aufgaben im Team. Konflikte innerhalb des Teams sollten frühestmöglich mit der Proseminarleiter:in besprochen werden.
- Alle in diesem Dokument aufgeführten und direkt daraus ableitbaren Arbeitsabläufe, Features und Anforderungen sind zwingend, vollständig und in angemessener Qualität für eine positive Beurteilung des Projekts umzusetzen!
- Die LVA- Leitung räumt sich das Recht ein, dass die von Studierenden erstellten Artefakte in anonymisierter Form für Lehre und Forschung verwendet werden können. Insbesondere können Projektergebnisse (und Teile davon) für Ausbildungszwecke eingesetzt werden.

Fristen, wichtige Termine und Abgaben

Frist / Zeitpunkt	Beschreibung
Bis 09.12.2024	Bilden Sie Teams (4-5 Personen) und legen Sie eine Ansprechperson fest.
Am 09.12.2024 im PS	Besprechung des Projekts, dessen Anforderungen und Bewertungskriterien. Festlegung der Teams. (Die LV-Leiter:innen unterstützen Sie gerne, falls es zu Problemen bei der Teameinteilung kommt)
Bis 16.12.2024 23:59 (Abgabe im GIT)	Erstellen Sie einen Projektplan bestehend aus - einem fachlichen UML Klassendiagramm und einem Sequenzdiagramm (als PDF in GIT ablegen). Das Sequenzdiagramm soll den Workflow für „kurzfristige Anpassungen“ abbilden. - Meilensteine (Anlegen in GitLab) - Issues und deren Verantwortlichkeiten (Anlegen in GitLab) Folgende Meilensteine müssen verpflichtend im Projektplan berücksichtigt werden: - Meilenstein 1: 13.01.2024 - Meilenstein 2: 20.01.2024 Der Inhalt der Meilensteine bleibt Ihnen überlassen. Gerne können Sie noch zusätzliche Meilensteine anlegen.
Am 16.12.2024 13.01.2024, 20.01.2024, jeweils 08:00	Es wird ein kontinuierlicher Projektfortschritt verlangt. Der Projektfortschritt wird anhand des GIT Projekts (z. B. Inhalt, Projektplan) durch die LV-Leiter:innen vor jedem Termin bewertet. Es zählt nur, was bis spätestens 08:00 Uhr am Tag der jeweiligen LV-Einheit in den Main-Branch gemerged ist. Bei zweimaliger Verletzung dieser Anforderung erfolgt eine negative Beurteilung des Projektteams bzw. der Studierenden.
Am 27.01.2024 08:00	Der Stand um 8:00 im GIT main-Branch wird für die Bewertung der Projektarbeit herangezogen. Bitte beachten Sie, dass Ihre Applikation mit der im Skeleton-Projekt definierten Maven-Konfiguration und den Befehlen - mvn spring-boot:run, sowie - npm start ausführbar sein muss. (vgl. Technische Anforderungen) Bitte vergessen Sie nicht, das fachliche UML-Klassendiagramm, das fachliche Sequenzdiagramm und den Projektplan ebenfalls in Ihrem GIT Repository abzulegen.
Bis 27.01.2024 im PS	Fertigen Sie eine Präsentation (max. 10 Minuten) mit interessanten Details zu ihrem System an, die im Proseminar präsentiert werden muss. Weitere Details zur finalen Präsentation und zum genauen Ablauf werden noch bekannt gegeben.

Hilfreiche Referenzen und Tools

Die folgenden Referenzen könnten bei der Bearbeitung der Projektarbeit hilfreich sein:

Allgemein

- VO Folien
- Kollaborationswerkzeuge (Jitsi, Slack, Discord, TeamViewer, Skype)
- Maven Dokumentation (<https://maven.apache.org/>)

GIT

- GIT Dokumentation (<https://git-scm.com/doc>)
- GIT Cheatsheet (<https://education.github.com/git-cheat-sheet-education.pdf>)

Spring Framework

- Dependency Injection (<https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-definition>)
- Bean Scopes (<https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-factory-scopes>)
- Deklaration von Beans (<https://docs.spring.io/spring-framework/docs/current/reference/html/core.html#beans-classpath-scanning>)
- Spring Data - Zugriff auf Daten (<https://spring.io/guides/gs/accessing-data-jpa/>)
- Spring Data – Zusammenspiel Repository und Service (<https://spring.io/blog/2011/02/10/getting-started-with-spring-data-jpa>)
- Spring Web (<https://docs.spring.io/spring-boot/reference/web/index.html>)
- Spring REST Tutorial (<https://spring.io/guides/tutorials/rest>)
- Baeldung REST Serie (<https://www.baeldung.com/rest-with-spring-series>)

React

- React-Dokumentation (<https://react.dev/>)
- React TypeScript Cheatsheet (https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/basic_type_example/)
- LearnXinYMinutes (<https://learnxinyminutes.com/docs/typescript/>)
- PrimeReact (<https://primereact.org/>)

API Tools

- Postman (<https://www.postman.com/>) oder jegliche Alternative

Dokumentation mittels JavaDoc

- Dokumentationskommentare mit JavaDoc (https://openbook.rheinwerk-verlag.de/javain-sel/23_004.html#u23.4)

UML-Modellierung

- UML-Spezifikation (<https://www.omg.org/spec/UML/About-UML/>)
- Hilfreiche UML-Webseite (<https://www.uml-diagrams.org/>)

Beispiele für UML-Tools

- Modelio (<https://www.modelio.org/>)

- Lucidchart (<https://www.lucidchart.com/pages/de>)
- Eclipse Papyrus (<https://www.eclipse.org/papyrus/>)
- UML Let (<https://www.umlet.com/>)
- Diagrams.net (<https://app.diagrams.net/>)

Dokumentation von Softwarearchitekturen

- Offizielle arc42 Webseite (<https://arc42.de/>)
- Beispielhafte arc42 Dokumentation (<https://www.dokchess.de/>)
- Java ist auch eine Insel, C. Ullенboom (<http://openbook.rheinwerk-verlag.de/javainsel/>)